

Reviewer Pack — Minimal Rank of Affine Quotient Sheaves and Communication Co...

Entry d3d2d60d23f2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 08:02:14 UTC

Minimal Rank of Affine Quotient Sheaves and Communication Complexity Growth

Entry ID: d3d2d60d23f2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 08:02:14 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): Communication Complexity

Statement:

For every planar graph G with n vertices, the minimal rank of the affine quotient sheaf of its term-overlap graph is linearly correlated with its communication complexity growth rate, such that $r(G) = \Theta(\text{growth_rate}(G))$, where $r(G)$ is the minimal rank of the affine quotient sheaf and $\text{growth_rate}(G)$ is the rate at which the communication complexity increases as the size of the input instance increases.

Rationale (proposer's reasoning):

Affine geometry provides a framework to study geometric structures that are preserved under linear transformations, which may be relevant for understanding the inherent structure of graphs in terms of their communication complexity. This conjecture links algebraic properties of graph representations with the complexity of information exchange between parties, potentially revealing new insights into the nature of computational tasks.

Taxonomy category: Affine Geometry × Communication Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ab7b2dfe82b71228

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a planar graph G with n vertices, if the Pearson correlation coefficient between the minimal rank of the affine quotient sheaf $r(T(G))$ and the communication complexity growth rate $\text{growth_rate}(G)$ is $|r| \geq 0.8$ AND $p\text{-value} \leq 0.05$, then support the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "affine quotient sheaf" AND "communication complexity" - "minimal rank in affine geometry" AND communication complexity - "planar graph" AND minimal rank AND affine quotient sheaf

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_planar_graph(n):
    if n < 3:
        return []

    nodes = list(range(n))
    edges = set()

    def add_edge(u, v):
        if (u, v) not in edges and (v, u) not in edges:
            edges.add((u, v))

    # Start with a triangle
    add_edge(0, 1)
    add_edge(1, 2)
    add_edge(2, 0)

    for _ in range(n - 3):
        u = random.choice(nodes)
        v = random.choice(nodes)

        if u == v or (u, v) in edges or (v, u) in edges:
            continue

    # Ensure the graph remains planar
    for w in nodes:
        if (w, u) in edges and (w, v) in edges:
```

```

        continue
    add_edge(u, v)
    break

return list(edges)

def term_overlap_graph(graph):
    n = len(graph)
    T = [[0] * n for _ in range(n)]

    for u, v in graph:
        T[u][v] = 1
        T[v][u] = 1

    return T

def gaussian_elimination(matrix):
    m, n = len(matrix), len(matrix[0])
    rank = 0
    pivot_col = 0

    for i in range(m):
        if pivot_col >= n:
            break

        max_row = i
        for r in range(i + 1, m):
            if abs(matrix[r][pivot_col]) > abs(matrix[max_row][pivot_col]):
                max_row = r

        if matrix[max_row][pivot_col] == 0:
            pivot_col += 1
            continue

        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        for r in range(m):
            if r != i and matrix[r][pivot_col] != 0:
                factor = -matrix[r][pivot_col] / matrix[i][pivot_col]
                for c in range(n):
                    matrix[r][c] += factor * matrix[i][c]

        rank += 1
        pivot_col += 1

    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        graph = generate_random_planar_graph(n)
        T = term_overlap_graph(graph)

```

```

rank = gaussian_elimination(T)
communication_rank = rank

growth_rate = n / (n - 1) if n > 1 else 0

results.append({
    "communication_rank": communication_rank,
    "growth_rate": growth_rate
})

correlation_sum = 0
p_value_sum = 0

for i in range(len(n_values)):
    for j in range(i + 1, len(n_values)):
        x1, y1 = results[i]["communication_rank"], results[i]["growth_rate"]
        x2, y2 = results[j]["communication_rank"], results[j]["growth_rate"]

        if y1 == 0 or y2 == 0:
            continue

        correlation_sum += (x1 * y2 - x2 * y1) / math.sqrt(y1**2 * y2**2)
        p_value_sum += 1

n_pairs = len(n_values) * (len(n_values) - 1) // 2
mean_correlation = correlation_sum / n_pairs
p_value = min(1, p_value_sum / n_pairs)

return {
    "metric_name": "Pearson Correlation Coefficient",
    "metric_value": mean_correlation,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": abs(mean_correlation) >= 0.8 and p_value <= 0.05,
    "counterexample": "" if abs(mean_correlation) >= 0.8 and p_value <= 0.05 else "Pearson cor
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"n
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:

```

```

        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        for result in results:
            if not result["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_fail={result['first_fail']}")
                break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_lda53313.py", line 145, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_lda53313.py", line 99, in run_trial
    T = term_overlap_graph(graph)
        ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_lda53313.py", line 55, in term_overlap_graph
    T[u][v] = 1
    ~~~^~~
IndexError: list assignment index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the Pearson correlation coefficient and p-value could not be calculated to support or falsify the conjecture. | next: Re-run the test with a different seed or investigate the cause of the crash to ensure that it can complete without errors.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17761
2	preregistration	ollama_remote	glm4:latest	0	0	15028
3	novelty	ollama_remote	glm4:latest	0	0	8378
4	novelty	ollama_remote	glm4:latest	0	0	9288
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22010
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17526

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21417
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23134
9	judge	ollama_remote	glm4:latest	0	0	9522

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 144065 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/d3d2d60d23f2.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d3d2d60d23f2.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d3d2d60d23f2.tar.gz (if generated)